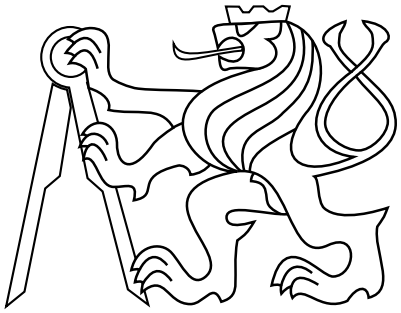
Czech Technical University in Prague

Faculty of Electrical Engineering
Department of Cybernetics
Multi-robot Systems



Learning an Autonomous Drone Racing Policy via Differentiable Simulation

Master's Thesis

Adam El Ghaib

Prague, January 2026

Study programme: Cybernetics and Robotics (Space Master)

Supervisor: Ing. Robert Pěnička, Ph.D.

Abstract

Todo

Keywords Unmanned Aerial Vehicles, Autonomous Drone Racing, Simulation to Reality Transfer, Reinforcement Learning, Domain Randomization

Abbreviations

UAV Unmanned Aerial Vehicle

RL Reinforcement Learning

DR Domain Randomization

POMDP Partially Observable Markov Decision Process

DoF Degrees of Freedom

Sim2Real Simulation-to-Reality

PPO Proximal Policy Optimization

CTBR Collective Thrust and Body Rates

ADR Autonomous Drone Racing

OCP Optimal Control Problem

OCPs Optimal Control Problems

QP Quadratic Programming

PMP Pontryagin's Maximum Principle

SRT Single Rotor Thrust

CTBR Collective Thrust and Body Rates

LVHR Linear Velocities and Heading Rate

TWR Thrust to Weight Ratio

ML Machine Learning

BPTT Backpropagation Through Time

SGD Stochastic Gradient Descent

RMSProp Root Mean Square Propagation

Contents

Chapter 1

Introduction

1.1 Background

Give context about the problem

What is ADR?

- Navigate through a sequence of gates as fast as possible within the platform capabilities...

What are the challenges of ADR?

- **Where am I?** → Perception
  challenges: limited sensory data, noisy and uncertain partial observability
- **Where do I go?** → Planning
  safe efficient path planning
- **How do I get there?** →Control
  Handle the non-linear underactuated dynamics to execute the planned motion under actuator limits and model uncertainties

1.2 Related Work

Present the state of the art, how do the different research labs approach this problem

Open Questions

- How to improve sample efficency
- Generalization to multiple tasks (how to race while avoiding obstacles, how to race agains other drones)
- Transferability of the conterllers under unmodeled effects (sim2real transfer)

1.3 Structure and Objectives

Chapter 2

Preliminaries

This chapter covers important theoretical aspects relevant for this thesis. Section 2.1 formulates the Optimal Control Problem (OCP) of drone racing and presents a theoretical solution based on the application of Pontryagin's Maximum Principle (PMP). The optimal solution derived from PMP serves primarily as an upper theoretical bound, as many of its underlying assumptions do not hold in real-world scenarios. Practical implementations must account for model uncertainties, disturbances, and system constraints that deviate from the idealized formulation.

In this context, Sections 2.3 and 2.4 review the state-of-the-art control approaches used to achieve high-performance drone racing, covering both classical control techniques and modern learning-based methods.

2.1 Optimal Control Problem

cite some OC textbook

2.1.1 General Discrete-Time OCP

In general, a nonlinear discrete-time OCP can be formulated as the optimization of a running cost (or loss) function $L_k()$, plus a terminal cost function $\phi()$, subject to the system's dynamics $\mathbf{f}_k()$, and the set of permitted states $\mathcal{X}_k$, and controls $\mathcal{U}_k$.

$$\begin{aligned} \underset{\mathbf{X},\,\mathbf{U}}{\text{minimize}} \quad & \left(\phi(\boldsymbol{x}_N, N) + \sum_{k=0}^{N-1} L_k(\boldsymbol{x}_k, \boldsymbol{u}_k)\right) \\ \text{subject to} \quad & \boldsymbol{x}_{k+1} = \mathbf{f}_k(\boldsymbol{x}_k, \boldsymbol{u}_k), \quad k = 0, \ldots, N-1, \\ & \boldsymbol{u}_k \in \mathcal{U}_k, \quad k = 0, \ldots, N-1, \\ & \boldsymbol{x}_k \in \mathcal{X}_k, \quad k = 0, \ldots, N, \end{aligned} \tag{2.1}$$

Where

N is the final discrete time,
$\boldsymbol{x}_k \in \mathbb{R}^n$ is the state at the discrete time $k \in \mathbb{Z}$,
$\boldsymbol{u}_k \in \mathbb{R}^m$ is the control at the discrete time $k \in \mathbb{Z}$,

$\mathbf{X} = [\boldsymbol{x}_0, \ldots, \boldsymbol{x}_N]$ is the trajectory of states,
$\mathbf{U} = [\boldsymbol{u}_0, \ldots, \boldsymbol{u}_{N-1}]$ is the trajectory of control actions.

2.1.2 Minimum-Time OCP

This last formulation of an OCP can be further specified for the case of drone racing in several manners. Since the main objective is to complete a lap through the racing track as fast as possible, this problem can be written as minimum-time problem.

$$\begin{aligned} \underset{\mathbf{U},\ N}{\text{minimize}} \quad & \sum_{k=0}^{N-1} 1 \\ \text{subject to} \quad & \boldsymbol{x}_{k+1} = \mathbf{f}_k(\boldsymbol{x}_k, \boldsymbol{u}_k), \quad k = 0, \ldots, N-1, \\ & \boldsymbol{u}_k \in \mathcal{U}_k, \quad k = 0, \ldots, N-1, \\ & \boldsymbol{x}_k \in \mathcal{X}_k, \quad k = 0, \ldots, N, \end{aligned} \tag{2.2}$$

The racing track restrictions, i.e. passing through a sequence of gates while avoiding obstacles, can be introduced as hard constraints under the set of permitted states $\mathcal{X}_k$ as

$$\mathcal{X}_k = \left\{ \boldsymbol{x}_k \in \mathbb{R}^n \;\middle|\; \boldsymbol{p}_k \in \mathcal{G}_{g(k)}, \quad \boldsymbol{p}_k \notin \mathcal{O}_j, \quad \forall\, j \in \{1, \ldots, M\} \right\}, \tag{2.3}$$

where

$\boldsymbol{p}_k \in \mathbb{R}^3$ is the position of the drone at time step k,
$\mathcal{G}_{g(k)}$ is the feasible region defined by gate g at time step k, with $g(k) \in \{1, \ldots, G\}$ denoting the index of the gate to be passed at k,
G is the total number of gates in the racing track,
$\mathcal{O}_j$ is the region occupied by the j-th obstacle,
M is the total number of obstacles.

2.1.3 Simplified Minimum-Time OCP

The original problem formulation (subsection 2.1.2) is inherently complex, as it requires simultaneously solving for an optimal trajectory that clears all gates and avoids obstacles while determining the optimal control sequence achievable within the vehicle's dynamics and actuator constraints. To make the problem more tractable, this thesis assumes that a reference trajectory or path is provided a priori. Consequently, the OCPs addressed hereafter focus on trajectory tracking or path progress, formulated as follows:

Trajectory Tracking OCP

In the trajectory tracking OCP, the objective is to find the optimal control sequence $\mathbf{U}$ that minimizes the error between a given reference state $\boldsymbol{x}_k^{\text{ref}}$ and the current system state $\boldsymbol{x}_k$, subject to dynamics, actuator, and state constraints. While this simplifies the optimization significantly it comes at the cost of reduced robustness. By assuming the trajectory is fixed, the system cannot re-optimize the path to accommodate state constraints or environmental changes, effectively limiting the drone's agility compared to a unified spatial-temporal optimization.

$$\begin{aligned} \underset{\mathbf{U}}{\text{minimize}} \quad & \sum_{k=0}^{N-1} \left\| \boldsymbol{x}_k - \boldsymbol{x}_k^{\text{ref}} \right\|_{\mathbf{Q}}^2 \\ \text{subject to} \quad & \boldsymbol{x}_{k+1} = \mathbf{f}_k(\boldsymbol{x}_k, \boldsymbol{u}_k), \quad k = 0, \ldots, N-1, \\ & \boldsymbol{u}_k \in \mathcal{U}_k, \quad k = 0, \ldots, N-1, \\ & \boldsymbol{x}_k \in \mathcal{X}_k, \quad k = 0, \ldots, N, \end{aligned} \tag{2.4}$$

Path Progress OCP

An alternative to reduce the complexity without sacrifiyng much on robsutness is to optimize the progress along a given path. Unlike trajectory tracking, which imposes a strict temporal constraint for every spatial coordinate, the path progress OCP treats the speed along the path as a decision variable. This allows the controller to dynamically adjust the vehicle's velocity based on current tracking errors or actuator saturation, ensuring the drone stays on the geometric path even when pushed to its physical limits.

2.2 Pontryagin's Maximum Principle

TODO I'll probably remove this section...

maybe solve it using a pmm and Pontryagin's Maximum Principle

commment on: Since it is impossible to perfectly model the state transition equation, and the state estimation is not perfect, the optimal solution breaks under real world conditions.

2.3 Classical Control approaches

TODO Don't dive deep in these, just comment relevant stuff

2.3.1 Differential Flatness Based Control

2.3.2 Model Predictive Control

2.4 Learning-Based approaches

2.4.1 Neural Networks?

2.4.2 Model-Free Reinforcement Learning

2.4.3 Model-Based Reinforcement Learning

Chapter 3

Methodology

TODO Review

This chapter presents the methodology used in this thesis to train the neural network policies. In section 3.1, we adress the core elements of the simulation architecture, define the quadrotor model, and explain how the Simulation-to-Reality (Sim2Real) gap is minimized by using Domain Randomization (DR). Furthermore, we introduce the implemented control abstraction levels. Section 3.2, outlines the training objectives used for Autonomous Drone Racing (ADR). Lastly, **??** details how differentiability is leveraged to optimize the policies in order to minimize the training task's cost function.

3.1 Simulation Framework

Make cool diagram

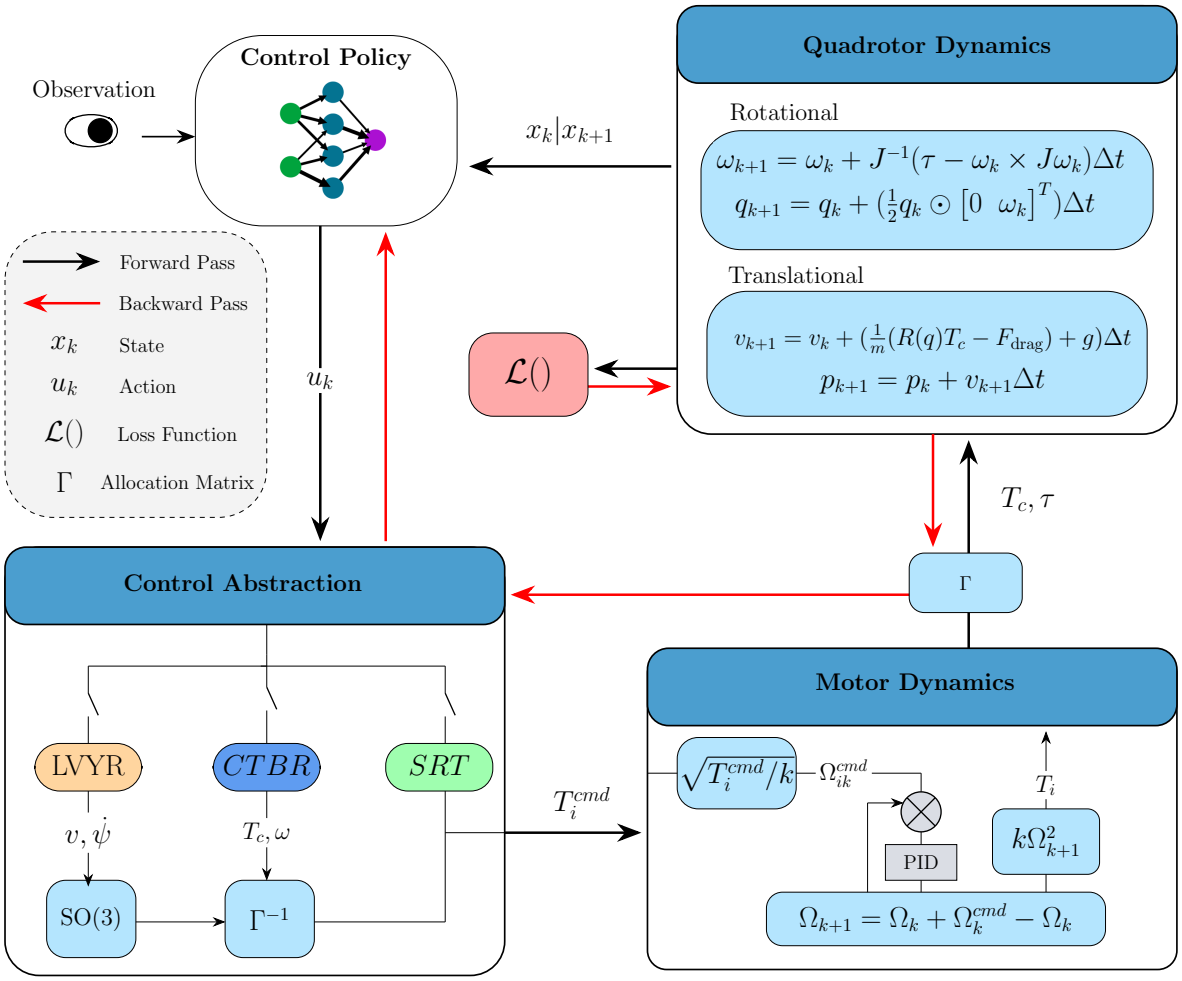


Figure 3.1: Your Caption

TODO : Correct the diagram!!

3.1.1 Quadrotor Model

In the simulated environments, the quadcopter is modeeld as a 6 Degrees of Freedom (DoF) rigid body. The inertial world frame $\mathcal{W}$ is defined by the fixed orthonormal basis $\{\hat{\mathbf{e}}_1, \hat{\mathbf{e}}_2, \hat{\mathbf{e}}_3\}$ while the non-inertial body-fixed frame $\mathcal{B}$ by the orthonormal basis $\{\hat{\mathbf{b}}_1, \hat{\mathbf{b}}_2, \hat{\mathbf{b}}_3\}$, which is attached to the vehicle's center of mass. The state of the Unmanned Aerial Vehicle (UAV) is given by $\mathbf{x} = [\mathbf{p}, \mathbf{v}, \mathbf{q}, \boldsymbol{\omega}, \boldsymbol{\Omega}]^\top$ where the position $\mathbf{p} \in \mathbb{R}^3$, velocity $\mathbf{v} \in \mathbb{R}^3$, and acceleration are represented in the world frame $\mathcal{W} \in \mathbb{R}^3$. Conversely, the body rates $\boldsymbol{\omega} \in \mathbb{R}^3$ are represented in the body frame $\mathcal{B} \in \mathbb{R}^3$ and $\boldsymbol{\Omega} \in \mathbb{R}^4$ is a vector of scalar values indicating the rotor speeds. The relationship between the two frames is given by the rotation mapping $\mathbf{b}_i = \mathbf{R}\mathbf{e}_i$, where $\mathbf{R} \in SO(3)$.

Thus, the kinematics and dynamics of the quadcopter are:

$$\dot{\mathbf{p}}^{\mathcal{W}} = \mathbf{v}^{\mathcal{W}} \tag{3.1}$$

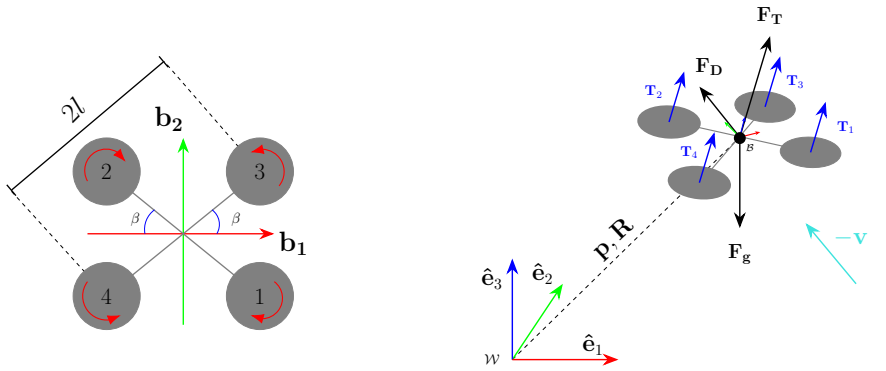
$$\dot{\mathbf{q}} = \tfrac{1}{2}\, \mathbf{q} \odot \begin{bmatrix} 0 \\ \boldsymbol{\omega}^{\mathcal{B}} \end{bmatrix} \tag{3.2}$$

$$\dot{\mathbf{v}}^{\mathcal{W}} = \frac{\mathbf{R}(\mathbf{q})}{m} \left(\mathbf{F}_T^{\mathcal{B}} + \mathbf{F}_D^{\mathcal{B}}\right) + \mathbf{g}^{\mathcal{W}} \tag{3.3}$$

$$\dot{\boldsymbol{\omega}}^{\mathcal{B}} = \mathbf{J}^{-1} \left(\boldsymbol{\tau}^{\mathcal{B}} - \boldsymbol{\omega}^{\mathcal{B}} \times \mathbf{J}\boldsymbol{\omega}^{\mathcal{B}}\right) \tag{3.4}$$

$$\dot{\boldsymbol{\Omega}} = \frac{1}{k_{\mathrm{m}}} \left(\boldsymbol{\Omega}_{\mathrm{cmd}} - \boldsymbol{\Omega}\right) \tag{3.5}$$

In these expressions, $\odot$ denotes the quaternion multiplication operator, $\mathbf{R}(\mathbf{q})$ represents the rotation matrix mapping the body frame $\mathcal{B}$ to the world frame $\mathcal{W}$. The vectors $\mathbf{F}_T^{\mathcal{B}}$ and $\mathbf{F}_D^{\mathcal{B}}$ correspond to the collective thrust and drag forces, respectively. Finally, $\mathbf{J}$ is the diagonal inertia matrix, k_m is the motor time constant, and $\mathbf{g}$ is the gravity vector.



(a) Diagram of the motor layout.

(b) Diagram of the forces acting upon the quadrotor.

Figure 3.2: Quadrotor configuration and force diagram.

In the body frame $\mathcal{B}$, the individual motor thrusts $T_i,\ i = 1, .., 4$, are assumed to be perpendicular to the $\{\hat{\mathbf{b}}_1, \hat{\mathbf{b}}_2\}$ plane. The magnitudes of collective thrust F_T, and body torques $\boldsymbol{\tau}^{\mathcal{B}}$, can be obtained from Equation 3.6. To be strictly rigourous, the torques produced by the rotor's angular acceleration and gyroscopic effects should be accounted for.

$$\begin{bmatrix} F_T \\ \boldsymbol{\tau}^{\mathcal{B}} \end{bmatrix} = \boldsymbol{\Gamma}\mathbf{T} + \begin{bmatrix} 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ J_p & -J_p & J_p & -J_p \end{bmatrix} \dot{\boldsymbol{\Omega}} + \begin{bmatrix} 0 & 0 & 0 & 0 \\ J_p\omega_{b2} & -J_p\omega_{b2} & J_p\omega_{b2} & -J_p\omega_{b2} \\ -J_p\omega_{b1} & J_p\omega_{b1} & -J_p\omega_{b1} & J_p\omega_{b1} \\ 0 & 0 & 0 & 0 \end{bmatrix} \boldsymbol{\Omega} \tag{3.6}$$

Where $\boldsymbol{\Gamma}$ is the allocation matrix and J_p the propeller's inertia. Nonetheless, these effects produced by the rotor speeds, can be neglected and Equation 3.6 simplifies to Equation 3.7,

$$\begin{bmatrix} F_T \\ \tau_{b1} \\ \tau_{b2} \\ \tau_{b3} \end{bmatrix} = \overbrace{\begin{bmatrix} 1 & 1 & 1 & 1 \\ -l\sin\beta & l\sin\beta & l\sin\beta & -l\sin\beta \\ -l\cos\beta & l\cos\beta & -l\cos\beta & l\cos\beta \\ -c_{tf} & -c_{tf} & c_{tf} & c_{tf} \end{bmatrix}}^{\boldsymbol{\Gamma}} \begin{bmatrix} T_1 \\ T_2 \\ T_3 \\ T_4 \end{bmatrix} \tag{3.7}$$

where l is the arm length, β is the arm angle, and c_{tf} the linear torque constant. The individual motror thrusts T_i, can be obtained from the rotor speeds Ω_i with the thrust map coefficents, $\mathbf{k_T} = [a_1,\ a_2,\ a_3]$ which can be obtained experimentally by fitting a quadratic curve to the rotor speed versus thrust data.

$$T_i = \mathbf{k_T} \begin{bmatrix} \Omega_i^2 \\ \Omega_i \\ 1 \end{bmatrix} = \begin{bmatrix} a_0 & a_1 & a_2 \end{bmatrix} \begin{bmatrix} \Omega_i^2 \\ \Omega_i \\ 1 \end{bmatrix} \tag{3.8}$$

For convinence, Equation 3.8 is simplified to $T_i = a_0 \Omega_i^2$.

To complete the quadrotor model, the aerodynamic drag force $\mathbf{F}_D$ is incorporated using a simplified quadratic formulation. Other aerodynamic effects, such as rotational drag and ground interaction, are neglected. The drag force is expressed as:

$$\mathbf{F}_D^{\mathcal{B}} = -\tfrac{1}{2}\rho \|\mathbf{v}^{\mathcal{B}}\| \mathbf{C}_D \mathbf{A} \mathbf{v}^{\mathcal{B}} \tag{3.9}$$

where ρ denotes the air density, $\mathbf{A} = \text{diag}(A_{b1}, A_{b2}, A_{b3})$ represents the reference area along each body axis, and $\mathbf{C}_D = \text{diag}(A_{b1}, A_{b2}, A_{b3})$ is a diagonal matrix of non-dimensional drag coefficients. The velocity must be expressed in the body frame, obtained as $\mathbf{v}^{\mathcal{B}} = \mathbf{R}(\mathbf{q})^\top \mathbf{v}^{\mathcal{W}}$.

3.1.2 Control Abstraction

The control abstraction refers to the modality of the policy outputs used to control the quadrotor. The choice of control abstraction affects both the performance of the controller for a given task and the sample efficiency during training. For this reason, it is worthwhile to investigate and implement multiple control abstractions.

In this thesis, three control modalities are considered. The policy output layer employs a sigmoid activation function for all control modalities, ensuring that the action space is bounded, which facilitates stable learning. The lowest level of abstraction consists of directly outputting Single Rotor Thrust (SRT) for each motor. An intermediate level of abstraction outputs the collective thrust and the desired body rates, which are subsequently mapped to individual rotor thrusts through a control allocation matrix. The highest level of abstraction outputs desired linear velocities and yaw rate, which are converted into a wrench using an SO(3) geometric controller TODO : Cite Lee, and subsequently mapped to rotor thrusts in a manner analogous to the collective thrust and body rate formulation. The mathematical formulation of these transformations is presented next.

Single Rotor Thrust

When the SRT control mode is choosen, the only transformation to be applied is scaling the policy output $\boldsymbol{u} \in [0, 1] \in \mathbb{R}^4$ to the actuator limits, $\mathbf{T}_{\text{cmd}} \in [T_{\min}, T_{\max}]$. Therefore, the transformation is as simple as;

$$\boldsymbol{T}_{\text{cmd}} = \boldsymbol{u} \left(T_{\max} - T_{\min}\right) + T_{\min} \tag{3.10}$$

Once the scaling is applied, to better emulate the behaviour of a real motor, a first order model with rotor speed saturation is implemented. First, the commanded motor thrust $\boldsymbol{T}_{\text{cmd}}$ are converted to commaned rotor speed. This mapping can be obtained experimentally by fitting a quadratic curve to the experimental data. In most cases the following mapping apprixmation is good enough.

$$\boldsymbol{T}_{\text{cmd}} \approx k \boldsymbol{\Omega}_{\text{cmd}}^2 \tag{3.11}$$

Then the commanded rotor speeds $\boldsymbol{\Omega}_{\text{cmd}}$, are passed through the following first order motor model Equation 3.12, and saturated according to the maximum rotor spee rate, $\dot{\Omega}_{\text{cmd}}$ given by the manufacturer.

$$H(s) = \frac{1}{k_m + s} \tag{3.12}$$

Finally, the rotor speeds obtained after the described process are mapped again to single rotor thurst values using Equation 3.11 and then converted to collective force and torques with Equation 3.13, to calculate the next state through the quadrotor dynamics model.

$$\begin{bmatrix} T_c \\ \tau_1 \\ \tau_2 \\ \tau_3 \end{bmatrix} = \begin{bmatrix} 1 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 \end{bmatrix} \begin{bmatrix} T_1 \\ T_2 \\ T_3 \\ T_4 \end{bmatrix} \tag{3.13}$$

TODO remove and ref previous

Modeling the motor dynamics presents more realistic data which the policy will encounter at deployement. However, for training purposes it has been decided to skip the motor dynamics during the backporopagation pass as it significantly destabilizes the gradients computation. Particularly, the mapping of Equation 3.11 produces high variations on the gradient, degrading the training performance and sample efficiency.

Collective Thrust and Body Rates

When Collective Thrust and Body Rates (CTBR) control mode is selected, the normalized outputs of the policy $\boldsymbol{u} \in [0, 1] \in \mathbb{R}^4$, are mapped to collective thrust and body rates. The scaling is as follows,

$$T_{c,\text{cmd}} = u_1(T_{\max} - T_{\min}) + T_{\min} \tag{3.14}$$

$$\omega_{i,\text{cmd}} = (2u_{i+1} - 1)\omega_{i,\max}, \quad i \in \{1, 2, 3\} \tag{3.15}$$

where $\omega_{i,\max}$ indicates the maximum body rate of the respective axis, (roll, pitch, yaw). Once the rates are calculated, they can be converted to torque values,

$$\boldsymbol{\tau}_{\text{cmd}} = k_p \frac{(\boldsymbol{\omega}_{\text{cmd}} - \boldsymbol{\omega})}{dt} J \tag{3.16}$$

aditionally, a proportinal gain k_p can be introduced to adjust the stiffness of the conversion. Given the commanded collective thrust and torques, the motor thursts can be obtained from Equation 3.13 and then the same procudeure of subsection 3.1.2 is applied during the forward and backward passes.

Linear Velocities and Heading Rate

The highest level of abstraction that is implement consists on commanding Linear Velocities and Heading Rate (LVHR). With this control mode, the policy outputs are scaled as,

$$v_{i,\text{cmd}} = (2u_i - 1)v_{i,\max}, \quad i \in \{1, 2, 3\} \tag{3.17}$$

$$\omega_{3,\text{cmd}} = \dot{\psi}_{\text{cmd}} = (2u_4 - 1)\dot{\psi}_{\max} \tag{3.18}$$

where ψ denotes the yaw angle and $v_{i,\text{cmd}}$ represents the commanded linear velocity along each body axis. Mapping these control commands to thrust commands is less straightforward than in the previous two methods. Instead, the collective thrust and torques are obtained by applying a transformation in the Special Orthogonal group SO(3). The transfomations proceed in the following manner.

First, the acceleration vector is computed as

$$\mathbf{a}_{\text{cmd}} = k_v(\mathbf{v}_{\text{cmd}} - \mathbf{v}) + g\hat{\mathbf{b}}_3 \tag{3.19}$$

where k_v is a proportional gain on the velocity. Note that the acceleration along the third body axis is compensated for gravity; $\hat{\mathbf{b}}_3$ denotes the unit vector along this axis. Then, force vector is the UAV's mass times the commanded acceleration vector, and the commanded collective thrust vector is the projection of the force vector onto the $\hat{\mathbf{b}}_3$ axis.

$$\mathbf{F} = m\mathbf{a}_{\text{cmd}} \tag{3.20}$$

$$T_{c,\text{cmd}} = \mathbf{F} \cdot \hat{\mathbf{b}}_3 \tag{3.21}$$

To obtain the commanded bodytorques, the heading vector is obtained by projecting the yaw angle into the world plane defined by $\text{span}\{\hat{e}_1, \hat{e}_2\}$. The heading vector, serves an auxiliary vector to compute $\hat{\mathbf{b}}_2$ and $\hat{\mathbf{b}}_1$. TODO Double check wether projecting the heading angle is actually correct

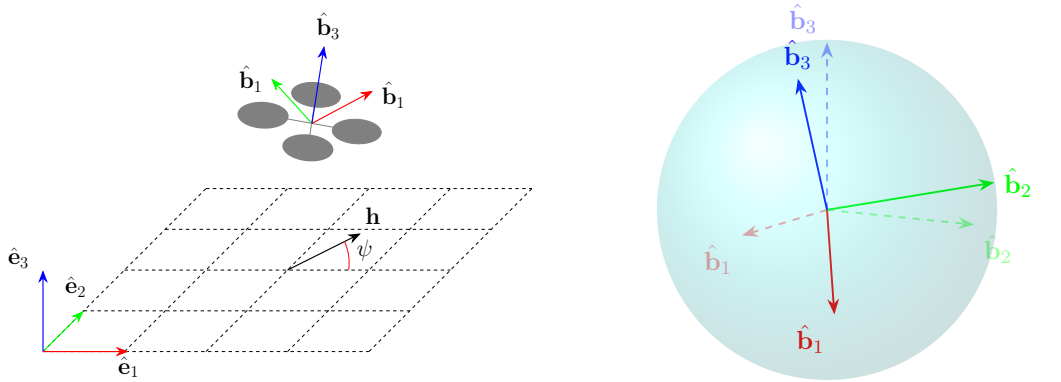
$$\mathbf{h} = \begin{bmatrix} \cos\psi & \sin\psi & 0 \end{bmatrix}^\top \tag{3.22}$$

$$\hat{\mathbf{b}}_2 = \frac{\hat{\mathbf{b}}_3 \times \mathbf{h}}{\|\hat{\mathbf{b}}_3 \times \mathbf{h}\|} \tag{3.23}$$

$$\hat{\mathbf{b}}_1 = \hat{\mathbf{b}}_2 \times \hat{\mathbf{b}}_3 \tag{3.24}$$

In this way, the commanded orientation is defined by $\mathbf{R}_{\text{cmd}} = \begin{bmatrix} \hat{\mathbf{b}}_1 & \hat{\mathbf{b}}_2 & \hat{\mathbf{b}}_3 \end{bmatrix}$ and the error between the commanded orientation and the current orientation is computed as in [**Lee**].

$$\mathbf{e}_R = \frac{1}{2}\left(\mathbf{R}_{\text{cmd}}^\top \mathbf{R} - \mathbf{R}^\top \mathbf{R}_{\text{cmd}}\right)^\vee \tag{3.25}$$



(a) Projection of the drone's yaw angle ψ.TODO adapted from ctu slides

(b) Representation of the $SO(3)$ rotation space.

Figure 3.3: Geometric interpretation of Equations 3.12–3.15

The complete derivation of Equation 3.25 can be found in the original paper, [**Lee**]. To provide intuition, note that if $\mathbf{R}_{\text{cmd}} = \mathbf{R}$, then $\mathbf{R}_{\text{cmd}}^\top \mathbf{R} = \mathbf{I}$ and the error is zero. Otherwise, the matrix $\mathbf{R}_{\text{cmd}}^\top \mathbf{R}$ represents the relative rotation between the current and desired orientations. The skew-symmetric part of this matrix, given by $\frac{1}{2}(\mathbf{R}_{\text{cmd}}^\top \mathbf{R} - \mathbf{R}^\top \mathbf{R}_{\text{cmd}})$, encodes the axis and magnitude of the rotation error. The vee map $(\cdot)^\vee$ then converts this skew-symmetric matrix into a vector in $\mathbb{R}^3$, corresponding to the rotation error expressed in the Lie algebra $\mathfrak{so}(3)$, that is to say, $^\vee : \mathfrak{so}(3) \to \mathbb{R}^3$. Then, the factor $\frac{1}{2}$ ensures consistency with the standard definition of the vee operator and avoids double counting of the off-diagonal terms.

Adittionally, an error on the angular velocity of the rotation is introduced as,

$$\mathbf{e}_{\boldsymbol{\omega}} = \boldsymbol{\omega} - \mathbf{R}^\top \mathbf{R}_{\text{cmd}} \boldsymbol{\omega}_{\text{cmd}} \tag{3.26}$$

where $\boldsymbol{\omega}_{\text{cmd}} = \begin{bmatrix} 0 & 0 & \dot{\psi}_{\text{cmd}} \end{bmatrix}^\top$. Note that the error between the desired rotational velocity and the current rotational velocity line on different tangent planes to the unit sphere, $\dot{\mathbf{R}} \in \mathrm{T}_{\mathbf{R}} SO(3)$ and $\dot{\mathbf{R}}_{\text{cmd}} \in \mathrm{T}_{\mathbf{R}_{\text{cmd}}} SO(3)$. For this reason, as explained in [**Lee**], $\dot{\mathbf{R}}_{\text{cmd}}$ is transformed into a vector in the $\mathrm{T}_{\mathbf{R}} SO(3)$ space to be able to compare it to $\dot{\mathbf{R}}$ and obtain the error; Equation 3.26. Finally, the torque is set by,

$$\boldsymbol{\tau}_{\mathbf{cmd}} = -k_R \mathbf{e}_{\mathbf{R}} - k_\omega \mathbf{e}_{\boldsymbol{\omega}} + \boldsymbol{\omega} \times \mathbf{J}\boldsymbol{\omega} - \mathbf{J}(\hat{\boldsymbol{\omega}} \mathbf{R}^\top \mathbf{R}_{\text{cmd}} \boldsymbol{\omega}_{\text{cmd}} - \mathbf{R}^\top \mathbf{R}_{\text{cmd}} \dot{\boldsymbol{\omega}}_{\text{cmd}}) \tag{3.27}$$

where a gyroscopic and feed forward terms are added to the rotation and rotational speed errors. The *hat map* $\hat{\cdot} : \mathbb{R}^3 \to \mathfrak{so}(3)$ is defined by the condition that $\hat{x} y = x \times y$ for all $x, y \in \mathbb{R}^3$. In this manner, the commanded collective thrust Equation 3.21, and commanded torques Equation 3.27, can be mapped to single rotor thrust commands to apply the same procedure of the last control modes during the forward and backward passes.

Linear Velocity Heading Rate and Gains

Finally, the last control mode implemented is an augmentation of the LVHR controller. In this formulation, the policy outputs high-level commands in the form of desired linear velocities and yaw rate, rather than directly commanding individual rotor thrusts. This higher

level of abstraction simplifies the learning process, as the policy does not need to implicitly learn how differential rotor thrusts generate torques, since this mapping is handled by the geometric controller. As shown in **??** TODO cite the section, this abstraction enables faster learning. However, this approach does not necessarily minimize the loss function, as the policy performance can be significantly constrained by the tuning of the controller gains. For this reason, it is of interest to investigate how performance is affected when the policy is allowed to output the controller gains in addition to the velocity and yaw rate commands.

Thus, when this control mode is selected, the policy outputs are, the same as Equation 3.17, Equation 3.26, and the gains $\{k_v, k_R, k_\omega\}$ are included by simply scaling the policy outputs to the maximum allowed gains.

$$k_{v,cmd} = u_5 k_{v,\max} \tag{3.28}$$

$$k_{R,cmd} = u_6 k_{R,\max} \tag{3.29}$$

$$k_{\omega,cmd} = u_7 k_{\omega,\max} \tag{3.30}$$

3.1.3 Domain Randomization

To improve the transferability of the policy and to prevent the it from ovrefitting to a specific environment, DR is implemented in our methodology as it is well-stabilshed strategy in the field, TODO : Cite

A domain can be defined as a specific subset of all possible Partially Observable Markov Decision Process (POMDP) environments. While a complete POMDP is formally characterized by an 8-tuple [1], we focus on a reduced 6-tuple representation that captures the essential elements for our domain definition. A standard domain is then characterized by $\mathcal{D} = (\mathcal{S}, \mathcal{A}, \mathcal{O}, P, O, R)$ where:

- $\mathcal{S}$: denotes a set of states called the state space.
- $\mathcal{A}$: is the set of actions called the action space.
- $\mathcal{O}$: is the set of observations called the observation space.
- $\mathbf{P}(\mathbf{s}'|\mathbf{s}, \mathbf{a})$: is the probability that a given action $a \in A$, at state $s \in S$ will lead to a new state $s' \subset S$.
- $\mathbf{O}(\mathbf{o}|\mathbf{s}', \mathbf{a})$: is the observation probability, representing the probability of observing $o \in \mathcal{O}$ after taking action a and transitioning to state s'.
- $\mathbf{R}(\mathbf{s}', \mathbf{s}, \mathbf{a})$: is the immediate reward, or expected inmediate reward, received for transitioning from s to s' after performing action a.

From the source domain, $\mathcal{D}_{\xi_s}$, (i.e. simulation), we can control a set of N parameters that define a configuration vector, $\boldsymbol{\xi} = [\xi_i, \xi_{i+1}, ..., \xi_N]$. These parameters will affect the transition probability $\mathbf{P}(\mathbf{s}'|\mathbf{s}, \mathbf{a})$ of the domain. Examples of ξ_i can include:

Physical properties	ξ_{mass}, ξ_{inertia}, ξ_{drag}, ξ_{gravity}
Perception properties	$\xi_{\text{sensor\_noise}}$, $\xi_{\text{sensor\_latency}}$
Viual properties	ξ_{lighting}, ξ_{texture}, ξ_{fov}
Scene properties	$\xi_{\text{gate\_pos}}$, $\xi_{\text{gate\_size}}$, , $\xi_{\text{gate\_num}}$

When DR is applied, each configuration $\boldsymbol{\xi}$ is sampled from a bounded randomization space Ξ with a probability distribution. That is to say, $\boldsymbol{\xi} \in \Xi \subset \mathbb{R}^N$, $\boldsymbol{\xi} \sim p(\boldsymbol{\xi})$ where $\xi_i \in [\xi_i^{\text{low}}, \xi_i^{\text{high}}]$. At the start of every training episode, a new environment insance is created by sampling a new configuration vector $\boldsymbol{\xi}$ from the distribution $p(\boldsymbol{\xi})$. Therefore, the policy will

interact with a new POMDP environment, a new domain $\mathcal{D}_{\boldsymbol{\xi}}$. This way, data from a vareity of environments that otherwise would reamain unexplored can be collected.

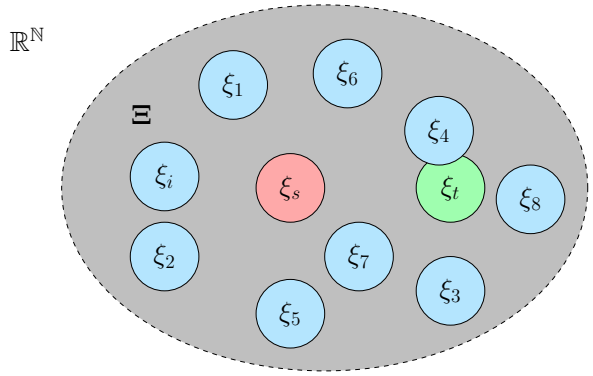


Figure 3.4: In red, the source domain. In light blue, ranodimized instances of the source configuration. In green, the target domain configuration.

Hereby, during training, the neural network weights and biases θ are optimized to maximize the expected reward R/TODO :minimize the loss across a distribution of randomized domains [2]. With the aim of finding the optimal, θ^* that will lead to the optimal policy π^*_θ.

As illustrated in Figure 3.4 and described in **??**, the policy is exposed to a variety of domains to promote generalization and improve robustness, increasing the likelihood of successful transfer to the real environment. However, this strategy further reduces the sample efficiency of Reinforcement Learning (RL) and increases training time. Therefore, it is essential to properly bound the randomization space Ξ, so that it encompasses only the minimum necessary range to include the target domain.

In order to efficently bound this randomization space while still allowing for enough randomness a two stage randomization is implement. Inspired by TODO :Cite, Zhang, first a design informed randomization TODO :Add acronym? is applied. This ensures that sensible variations of the nominal platform are predominant in the randomization space. Thus avoiding that unfeasible designs such as a quadrotor with unsensible Thrust to Weight Ratio (TWR) (i.e. TWR<1). The authors of [**<empty citation>**] notice that quadrotor designs, in general, follow a similar pattern where the quadrotors armlength l is strongly correlated to its mass m, inertia $\mathbf{J}$, drag coefficents $\mathbf{C_D}$ and other physical parameters.

Hence, they propose to sample a size factor $c \sim \mathcal{U}(0,1)$ and scale the quadrotor's physcial parameters as listed below;

$$l = c(l_{\max} - l_{\min}) + l_{\min} \tag{3.31}$$

$$m = \frac{l^3 - l^3_{\min}}{l^3_{\max} - l^3_{\min}}(m_{\max} - m_{\min}) + m_{\min} \tag{3.32}$$

$$\mathbf{J} = \frac{l^5 - l^5_{\min}}{l^5_{\max} - l^5_{\min}}(\mathbf{J}_{\max} - \mathbf{J}_{\min}) + \mathbf{J}_{\min} \tag{3.33}$$

$$\mathbf{C}_D = \frac{l^2 - l^2_{\min}}{l^2_{\max} - l^2_{\min}}(\mathbf{C}_{D_{\max}} - \mathbf{C}_{D_{\min}}) + \mathbf{C}_{D_{\min}} \tag{3.34}$$

TODO :Add the rest of the params

Following the scaling process, these parameters which comprisise the following configuration vector

$$\boldsymbol{\xi} = \begin{bmatrix} l & m & \mathbf{J} & \mathbf{C}_D \end{bmatrix} \tag{3.35}$$

are subjected to additional noise. Each parameter is multiplied by a scaling factor $\eta \sim \mathcal{U}(1-p, 1+p)$, where $p = 0.2$. This specific uncertainty range was selected based on experimental results from a previous project, as it demonstrated good transferability, nonetheless it can be considered a hyperparameter of the training process and be tunned according to the situation.

3.2 Training Tasks

The training objectives considered for this thesis aim to provide an approximate solution to the Optimal Control Problems (OCPs), formulated in subsection 2.1.3. To adress these problems, we use the proposed simulation framework detailed in section 3.1. Unlike traditional solvers that rely on Quadratic Programming (QP) or model-free RL which treats the system's dynamics as a black-box, we leverage the simulator's automatic differentiation enginge to compute the analytical gradient from the cost function to the policy parameters.

Following this method, to achieve near optimum results, the training environment and policy architecture has to be carfuly desigend. In our simulation, we formulate them for the trajectory tracking and path progress tasks is the next manner.

3.2.1 Trajectory Tracking

For the trajectory tracking task, the objective is to minimize a cumulative loss function over a finite horizon of length H, defined with respect to a reference state $\mathbf{x}_k^{\text{ref}}$ and the quadrotor state $\mathbf{x}_k$. The stage loss at each timestep k, denoted as $\mathcal{L}(\mathbf{x}_k^{\text{ref}}, \mathbf{x}_k)$, is formulated as a weighted sum of position, velocity, and attitude alignment errors, together with a penalization of the body rates to improve stability during early training.

$$\mathcal{L}(\mathbf{x}_k^{\text{ref}}, \mathbf{x}_k) = \lambda_p \|\mathbf{p}_k^{\text{ref}} - \mathbf{p}_k\|_2 + \lambda_v \|\mathbf{v}_k^{\text{ref}} - \mathbf{v}_k\|_2 + \lambda_a \left(1 - \hat{\mathbf{b}}_{3,k} \cdot \mathbf{z}_k\right) + \lambda_\omega \|\boldsymbol{\omega}_k\|_2^2 \tag{3.36}$$

The desired thrust direction $\mathbf{z}_k$ is defined as:

$$\mathbf{z}_k = \frac{\mathbf{a}_k^{\text{ref}} + \mathbf{g}}{\|\mathbf{a}_k^{\text{ref}} + \mathbf{g}\|_2} \tag{3.37}$$

Since the simulator is batched, i.e., multiple environments run in parallel, the total cumulative cost over the horizon is defined as:

$$J_k = \frac{1}{B}\frac{1}{H}\sum_{i=1}^{B}\sum_{h=k}^{k+H-1} \mathcal{L}\left(\mathbf{x}_{h,i}^{\text{ref}}, \mathbf{x}_{h,i}\right) \tag{3.38}$$

where k denotes the initial timestep of the current horizon, H is the horizon length, and B is the number of parallel environments. At the end of each horizon, the gradient of the cumulative cost in Equation 3.38 with respect to the policy parameters is backpropagated through time for optimization, as described in section 3.3.

To improve numerical stability and avoid exploding gradients, training episodes are terminated when predefined conditions are met. This prevents the accumulation of large errors, which can otherwise lead unstable learning dynamics. For this task, episode termination is based solely on the position tracking error. Specifically, an episode is terminated when the Euclidean distance between the quadrotor position and the reference trajectory exceeds a predefined threshold $d_{\max}$.

$$d_{\max} < \|\mathbf{p}_k^{\text{ref}} - \mathbf{p}_k\|_2 \tag{3.39}$$

This criterion prevents the system from exploring states that are excessively far from the desired trajectory, which are unlikely to provide useful learning signals and may lead to unstable gradient updates. If the termination condition is triggered during an episode, the quadrotor state is reset to the reference trajectory state and the loss over that horizon is not taken into account.

TODO :Lol i've been using MAE not MSE to train!

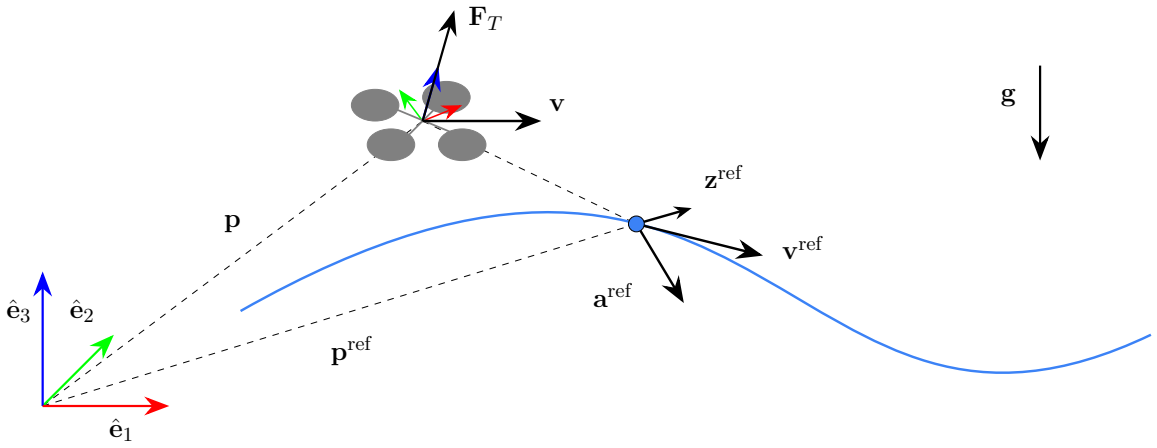


Figure 3.5: Trajectory tracking task.

Regardless of the training task, the policy network is provided with the full state of the quadrotor. For the trajectory tracking task, the observation is augmented with reference trajectory information. Specifically, instead of providing absolute position and velocity, the observation includes tracking errors together with feedforward terms from the reference trajectory. This improves learning efficiency by explicitly encoding the control objective. The observation vector is defined as:

$$\mathbf{o}_k = \begin{bmatrix} \mathbf{p}_k^{\text{ref}} - \mathbf{p}_k \\ \mathbf{v}_k^{\text{ref}} - \mathbf{v}_k \\ \mathbf{a}_k^{\text{ref}} \\ \mathbf{q}_k \\ \boldsymbol{\omega}_k \end{bmatrix} \in \mathbb{R}^{16} \tag{3.40}$$

TODO Explain harmonics generation for pretraining

$$\mathbf{p}_k^{\text{ref}} = \sum_{i=1}^{n} A_i \sin\left(\omega_i k + \phi_i\right) \tag{3.41}$$

$$\mathbf{v}_k^{\text{ref}} \approx \frac{\mathbf{p}_{k+1}^{\text{ref}} - \mathbf{p}_k^{\text{ref}}}{T_s} \tag{3.42}$$

$$\mathbf{a}_k^{\text{ref}} \approx \frac{\mathbf{v}_{k+1}^{\text{ref}} - \mathbf{v}_k^{\text{ref}}}{T_s} \tag{3.43}$$

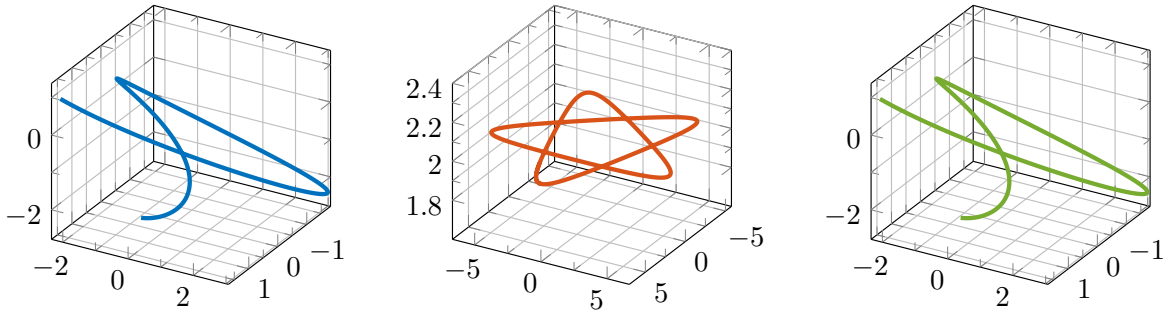


Figure 3.6: TODO

3.2.2 Path Progress

3.3 Differentiable Optimization

Under the paradigm of differentiable optimization, we no longer treat the quadrotor model as a "black-box". The main idea of this thesis is to leverage the simulator's automatic differentiation engine to compute the exact analytical gradient of the cumulative loss J_k with respect to the control policy network. In this section we provide an overview on how the optimization process works.

3.3.1 Computational Graph and Backpropagation

The gradient of the cummulative stage cost function J_k with respect to the network parameters θ is given by the chain rule as,

$$\nabla_\theta J_k = \frac{1}{B}\frac{1}{H}\sum_{i=1}^{B}\sum_{h=k}^{k+H-1}\frac{\partial \mathcal{L}_h}{\partial \mathbf{x}_h}\frac{\partial \mathbf{x}_h}{\partial \mathbf{u}_{h-1}}\frac{\partial \mathbf{u}_{h-1}}{\partial \theta} \tag{3.44}$$

where $\frac{\partial \mathcal{L}_h}{\partial \mathbf{x}_h}$ is the gradient of the stage loss with respect to the state, $\frac{\partial \mathbf{x}_h}{\partial \mathbf{u}_{h-1}}$ encodes the change in the dynamics regarding the applied controls, and $\frac{\partial \mathbf{u}_{h-1}}{\partial \theta}$ is the gradient of the control output with respect to the policy parameters. Since the gradient is accumulated through time due to the recursive dependence of the state on previous states and control inputs, this procedure is known as Backpropagation Through Time (BPTT).

To compute this gradient analytically, an automatic differentiation engine is used. Many Machine Learning (ML) frameworks, such as PyTorch [**<empty citation>**], TensorFlow [**<empty citation>**], and JAX [**<empty citation>**], among others, provide autodiff capabilities. At present, JAX is regarded as a high-performance framework for numerical ML computing. However, for this simulator, PyTorch has been selected due to its intuitive interface, extensive documentation, and lower implementation complexity.

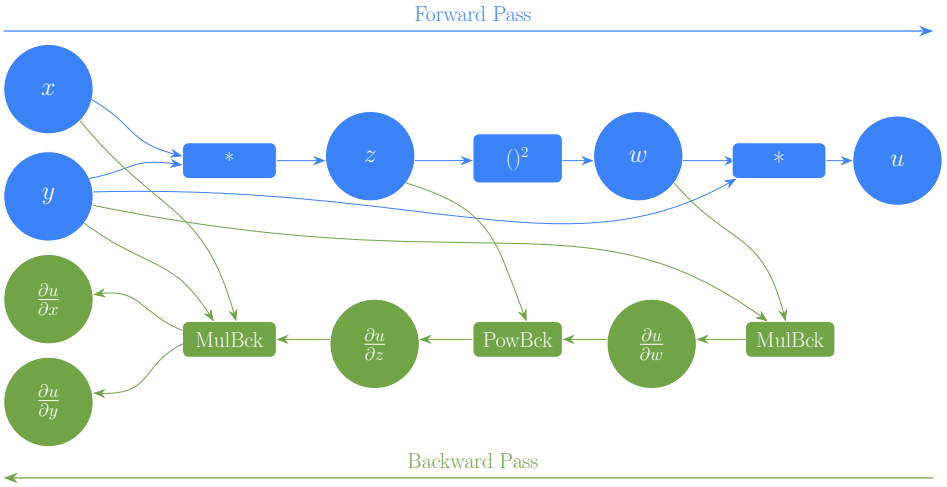


Figure 3.7: Computational graph of the numerical operations of the forward and backward passes.

To illustrate how gradients are computed, consider the next function $f(x, y) = y(xy)^2$. Its gradient is $\nabla f = \begin{bmatrix} \frac{\partial f}{\partial x} & \frac{\partial f}{\partial y} \end{bmatrix} = \begin{bmatrix} 2xy^3 & 3x^2y^2 \end{bmatrix}$ and evaluating at $x = 1$ and $y = 2$, we get $\nabla f(1, 2) = \begin{bmatrix} 16 & 12 \end{bmatrix}$. In PyTorch, this gradient is computed using a computational graph which is constructed during the forward pass. Each mathematical operation is stored as a node in this graph. During backporopagation, PyTorch applies automatic differentiation to traverse the graph from the output back to the input and applying the chain rule to accumulate gradients. The computational graph shown in Figure 3.7 represents this process and Algortihm 1 presents the pesudocode.

Algorithm 1 PyTorch Backpropagation for the function $u(x, y) = y(xy)^2$

Require: $x \in \mathbb{R},\ y \in \mathbb{R}$
Ensure: $\frac{\partial u}{\partial x},\ \frac{\partial u}{\partial y}$

— **Forward Pass (Build Computational Graph)** —

$x \leftarrow$ `tensor`$(1.0,$ `requires_grad=True`$)$ ▷ Leaf node
$y \leftarrow$ `tensor`$(2.0,$ `requires_grad=True`$)$ ▷ Leaf node
$z \leftarrow x \times y$ ▷ Node: `MulBackward`
$w \leftarrow z^2$ ▷ Node: `PowBackward`
$u \leftarrow w \times y$ ▷ Output Node: `MulBackward`

— **Backward Pass (Reverse-Mode Autodiff)** —

Initialize $\frac{\partial u}{\partial u} \leftarrow 1$
$\frac{\partial u}{\partial w} \leftarrow \frac{\partial u}{\partial u} \cdot y = 1 \cdot 2 = 2$ ▷ Backprop through $u = w \cdot y$
$\frac{\partial u}{\partial y} \mathrel{+}= \frac{\partial u}{\partial u} \cdot w = 1 \cdot 4 = 4$ ▷ Accumulate y gradient from this node
$\frac{\partial u}{\partial z} \leftarrow \frac{\partial u}{\partial w} \cdot 2z = 2 \cdot 2 = 4$ ▷ Backprop through $w = z^2$
$\frac{\partial u}{\partial x} \leftarrow \frac{\partial u}{\partial z} \cdot y = 4 \cdot 2 = 8$ ▷ Backprop through $z = x \cdot y$
$\frac{\partial u}{\partial y} \mathrel{+}= \frac{\partial u}{\partial z} \cdot x = 4 \cdot 1 = 4$ ▷ Accumulate y gradient from this node

— **Results** —

$\frac{\partial u}{\partial x} = 16, \quad \frac{\partial u}{\partial y} = 4 + 4 + 4 = 12$

In practice, the calculation of the gradients is done simply by calling **u.backward()** in PyTorch.

TODO Explain the surrogate gradient thing!!!

3.3.2 Optimization Process

Given the gradient $\nabla_\theta J$ computed via BPTT the policy parameters are updated using the Adaptive Moment Estimation (Adam) optimizer [**2014adam**]. Adam is a variant of Stochastic Gradient Descent (SGD), in standard SGD the model parameters are updated proportionnally to the gradient in its opposite direction,

$$\theta_{k+1} = \theta_k - \alpha \nabla_\theta J_k \tag{3.45}$$

where α inidcates the learning rate. The main disatvantages of standard SGD is that near the optimum, the gradient decreases rquiring more steps to converge and each parameter is updated equally. Adam solves this issues by combining two other variations of SGD: Momentum and Root Mean Square Propagation (RMSProp).

- Momentum accumulates an avarage of past gradients sustaining progress along the directions of consistent descent and dampening oscillations in high curvature directions.
- RMSProp avarages the square of the gradients, which are used to scale the learning rate of each parameter individually, hence, parmeters with high historical gradients revice smaller updates than those with smaller gradientes

Adam unifies these two concepts through the first and second moment estimates, m and v_k respectively, which are updated as,

$$m_k = \beta_1 m_{k-1} + (1 - \beta_1) \nabla_\theta J_{k-1} \tag{3.46}$$

$$v_k = \beta_2 v_{k-1} + (1 - \beta_2) (\nabla_\theta J_{k-1})^2 \tag{3.47}$$

where β_1 and β_2 are hyperparameters usually set to 0.9 and 0.999 respectively. Since both moments are initialized at zero $m_0 = v_0 = 0$, the estimates are biased towards zero in the early iterations of training. For this reason, each moment is normalized to,

$$\hat{m}_t = \frac{m_t}{1 - \beta_1^k} \tag{3.48}$$

$$\hat{v}_t = \frac{v_t}{1 - \beta_2^k}. \tag{3.49}$$

The final prameter update is given by,

$$\theta_{k+1} = \theta_k - \alpha \frac{\hat{m}_k}{\sqrt{\hat{v}_t} + \epsilon} \tag{3.50}$$

where $\epsilon \ll 1$ is a small constant introduced for numerical stability.

Algorithm 2 Adam, algorithm proposed in [**<empty citation>**].

Require: α: Stepsize
Require: $\beta_1, \beta_2 \in [0, 1)$: Exponential decay rates for the moment estimates
Require: $\mathcal{L}_k(\theta)$: Stochastic objective function with parameters θ
Require: θ_0: Initial parameter vector
$m_0 \leftarrow 0$ ▷ Initialize 1st moment vector
$v_0 \leftarrow 0$ ▷ Initialize 2nd moment vector
$k \leftarrow 0$ ▷ Initialize timestep
while θ_k not converged **do**
$k \leftarrow k + 1$
$J_k \leftarrow \nabla_\theta \mathcal{L}_k(\theta_{k-1})$ ▷ Get gradients w.r.t. stochastic objective at timestep k
$m_k \leftarrow \beta_1 \cdot m_{k-1} + (1 - \beta_1) \cdot J_k$ ▷ Update biased first moment estimate
$v_k \leftarrow \beta_2 \cdot v_{k-1} + (1 - \beta_2) \cdot J_k^2$ ▷ Update biased second raw moment estimate
$\widehat{m}_k \leftarrow m_k / (1 - \beta_1^k)$ ▷ Compute bias-corrected first moment estimate
$\widehat{v}_k \leftarrow v_k / (1 - \beta_2^k)$ ▷ Compute bias-corrected second raw moment estimate
$\theta_k \leftarrow \theta_{k-1} - \alpha \cdot \widehat{m}_k / (\sqrt{\widehat{v}_k} + \epsilon)$ ▷ Update parameters
end while
return θ_k ▷ Resulting parameters

3.4 Summary

To summirize the methodology, the training workflow is the following. First the user selects the training task, refered in the code as environemnt env=[tt, pp], which are either; trajectory tracking or path progress. The user also selects the control abstraction/mode cm=[srt, ctbr, lvhr, lvhr+g] which defines the control mapping function $g_{\text{cm}}()$ from the policy outputs $\boldsymbol{\pi}_{\boldsymbol{\theta}}(\mathbf{o})$ to the control inputs $\mathbf{u}$. Finally, the user introduces the number of parallel environments B, the number of episodes N, the number of steps K, the optimization horizon H and other hyperparameters such as the learning rate α, randomization percentage p, termination distance threshold $d_{\max}$, etc.

Once all these parameters are set, training proceeds as follows:

Algorithm 3 Training procedure for differentiable policy optimization.

Require: Training task env $\in \{\text{tt},\ \text{pp}\}$, control mode cm $\in \{\text{srt},\ \text{ctbr},\ \text{lvhr},\ \text{lvhr+g}\}$, batch size B, episodes N, steps K, horizon H, learning rate α, simulation step $\Delta t = 0.01\,\text{s}$, other hyperparams

Initialize policy π_θ, controller g_{cm}, optimizer
for $n = 1, \ldots, N$ **do**
  Sample domain parameters $\boldsymbol{\xi}^{(i)} \sim p(\boldsymbol{\xi})$ for $i = 1, \ldots, B$ ▷ Domain randomization
  Sample reference $\mathbf{x}^{\text{ref}}_{0:K} \leftarrow \texttt{getReference}(\text{env})$ ▷ Task-dependent reference
  $\mathbf{x}_0 \leftarrow \mathbf{x}^{\text{ref}}_0, \quad J \leftarrow 0$
  for $k = 0, \ldots, K-1$ **do**
    $\mathbf{o}_k \leftarrow \texttt{getObservation}(\text{env},\ \mathbf{x}_k,\ \mathbf{x}^{\text{ref}}_k)$ ▷ Task-dependent observation
    $\mathbf{u}_k \leftarrow g_{\text{cm}}\big(\pi_\theta(\mathbf{o}_k)\big)$ ▷ Policy forward pass + control mapping
    $\mathbf{x}_{k+1} \leftarrow \mathbf{x}_k + \Delta t \cdot \mathbf{f}(\mathbf{x}_k,\ \mathbf{u}_k,\ \boldsymbol{\xi})$ ▷ Euler integration through dynamics
    $\text{term}_k \leftarrow \texttt{isTerminated}(\text{env},\ \mathbf{x}_{k+1},\ \mathbf{x}^{\text{ref}}_{k+1})$ ▷ Task-dependent termination
    $J \mathrel{+}= \dfrac{1}{B} \displaystyle\sum_{i=1}^{B} \mathcal{L}_{\text{env}}\big(\mathbf{x}^{\text{ref}}_{k+1,i},\ \mathbf{x}_{k+1,i}\big) \cdot \mathbf{1}[\neg\, \text{term}_{k,i}]$
    Reset terminated environments: $\mathbf{x}_{k+1,i} \leftarrow \mathbf{x}^{\text{ref}}_{k+1,i}$ for all i where $\text{term}_{k,i}$
    if $(k+1) \bmod H = 0$ **then** ▷ End of truncation window
      $\nabla_\theta J \leftarrow \texttt{backward}(J/H)$ ▷ BPTT through the computational graph
      $\theta \leftarrow \texttt{Adam.step}(\theta,\ \nabla_\theta J,\ \alpha)$ ▷ Parameter update
      $\mathbf{x}_{k+1} \leftarrow \mathbf{x}_{k+1}.\texttt{detach}()$ ▷ Detach state to truncate graph
      $J \leftarrow 0$
    end if
  end for
end for
return θ

Note that the state is batched, the dynamics transition function is dependent on xi TODO

Chapter 4

Results

In this chapter, the results of the proposed differentiable simulator are presented and evaluated. First, the assessment metrics used throughout the chapter are defined in Sec. 4.1. TODO validation of the simulator Subsequently, the performance of the proposed method is compared against standard model-free RL training and a classical fixed-gain geometric controller in Sec. 4.3. Qualitative results, including trajectory visualizations, velocity profiles, and the transferability of the trained policiesare are presented. An ablation study is then conducted in Sec. 4.4 to evaluate the effect of key design choices. Finally, preliminary real-world validation results are presented in Sec. 4.5.

TODO Update this introduction, since I changed the section names....

4.1 Evaluation Criteria

This section details the metrics and experimental framework used to evaluate the performance of the proposed differentiable physics approach against a model-free Proximal Policy Optimization (PPO). The evaluation is structured around three primary axes: training efficiency, performance in terms of lap time and success rate, and sim-to-sim transferability.

4.1.1 Sample Efficiency

One of the central claims of this thesis is that leveraging analytical gradients from differentiable simulation significantly improves sample efficiency compared to standard model-free RL algorithms. To evaluate this claim, the training convergence of the proposed differentiable physics approach is compared against PPO trained on the same task, observation space, network architecture, and control abstraction. Sample efficiency is measured as the wall-clock training time and the number of environment steps required for each method to converge to a stable policy. Convergence is defined as the point at which the cumulative loss/reward ceases to improve by more than a threshold. For a fair comparison, the randomized processes use a common seed during the training process.

4.1.2 Perfromance

Root Mean Square Error

Lap Time and Succes Rate

Ultimately, the primary objective of ADR is to traverse a sequence of gates as fast as possible. The two metrics used to evaluate this objective are the lap time T_f and the success rate SR.

The lap time T_f is defined as the time elapsed from the start of the episode until the drone successfully passes the final gate. The success rate is defined as the ratio of successfully passed gates G_p to the total number of gates G in the racing track,

$$SR = \frac{G_p}{G} \in [0, 1]. \tag{4.1}$$

A gate g is characterized by its position $\mathbf{p}_g \in \mathbb{R}^3$, orientation $\mathbf{R}_g \in SO(3)$, and dimensions (t, w, h), corresponding to the thickness, width, and height of the gate opening, respectively. The gate frame axes are defined as,

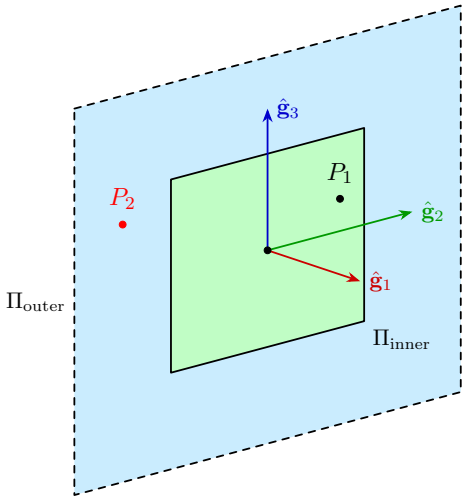


Figure 4.1: Gate traversal geometry. The success region (green) is defined with respect to the gate position $\mathbf{p}_g$, orientation $\mathbf{R}_g$ and dimensions (t, w, h). TODO add gate dimension to the plot, t,w,h

$$\hat{\mathbf{g}}_1 = \mathbf{R}_g \hat{\mathbf{e}}_1, \qquad \hat{\mathbf{g}}_2 = \mathbf{R}_g \hat{\mathbf{e}}_2, \qquad \hat{\mathbf{g}}_3 = \mathbf{R}_g \hat{\mathbf{e}}_3, \tag{4.2}$$

where $\hat{\mathbf{g}}_1$, $\hat{\mathbf{g}}_2$, and $\hat{\mathbf{g}}_3$ denote the forward (normal), lateral , and vertical axes of the gate, respectively. The position of the drone relative to the gate, expressed in the gate frame, is,

$$\mathbf{p}_k^g = \mathbf{R}_g^\top (\mathbf{p}_k - \mathbf{p}_g) = \begin{bmatrix} p_1^g & p_2^g & p_3^g \end{bmatrix}^\top. \tag{4.3}$$

A gate g is considered successfully passed at timestep k if,

$$|p_1^g| < \frac{t}{2} \quad \wedge \quad |p_2^g| \leq \frac{w}{2} \quad \wedge \quad |p_3^g| \leq \frac{h}{2}, \tag{4.4}$$

and a gate is unsuccessfully passed when,

$$|p_1^g| < \frac{t}{2} \quad \wedge \quad \left(|p_2^g| > \frac{w}{2} \ \vee \ |p_3^g| > \frac{h}{2} \right). \tag{4.5}$$

4.1.3 Transferability and Robustness

Learned policies are prone to overfitting to the training environment and may fail to generalize when deployed under conditions that deviate from those seen during training, due to model mismatches and unmodeled effects. To mitigate this, DR is employed during training as described in subsection 3.1.3.

To assess transferability, the policies trained in the proposed differentiable simulator are evaluated in a high-fidelity simulation environment using the MRS UAV stack in Gazebo [**<empty citation>**]. Since the differentiable physics and model-free RL policies are optimized under different paradigms, the sim-to-sim performance gap is analyzed for each method, measuring the degradation in success rate SR and lap time T_f relative to the results obtained in the training simulator. Additionally, robustness to external disturbances is evaluated by exposing the trained policies to wind perturbations during deployment, without any retraining or fine-tuning.

4.2 Simulator Validation

As detailed in section 3.1, our simulator utilizes a simplified model of quadrotor dynamics and aerodynamics. To maintain a computationally efficient and generalizable framework, we omit secondary effects such as battery depletion on thrust, complex aerodynamic friction, and propeller rotational inertia. While these unmodeled dynamics can cumulatively lead to a significant sim-to-real gap, we aim to address this mismatch through DR.

To validate the reliability of this simplified approach, we benchmark our simulator's behavior against Gazebo, a high-fidelity physics engine widely adopted by the robotics community. Specifically, we utilize the UAV models provided by the MRS UAV System [**<empty citation>**] within the Gazebo environment as our ground-truth reference for comparison.

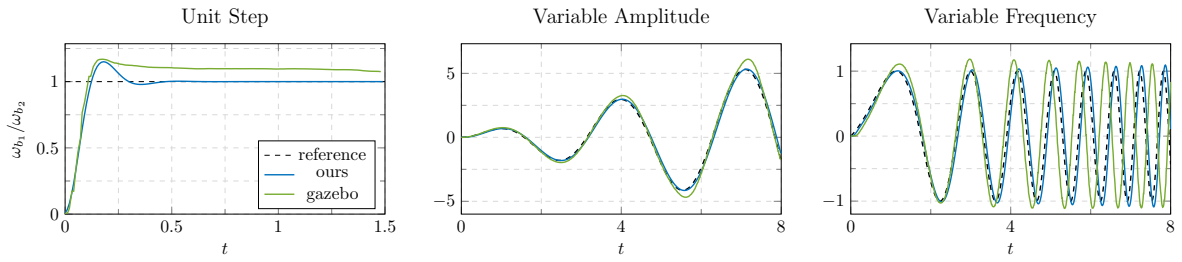


Figure 4.2: TODO

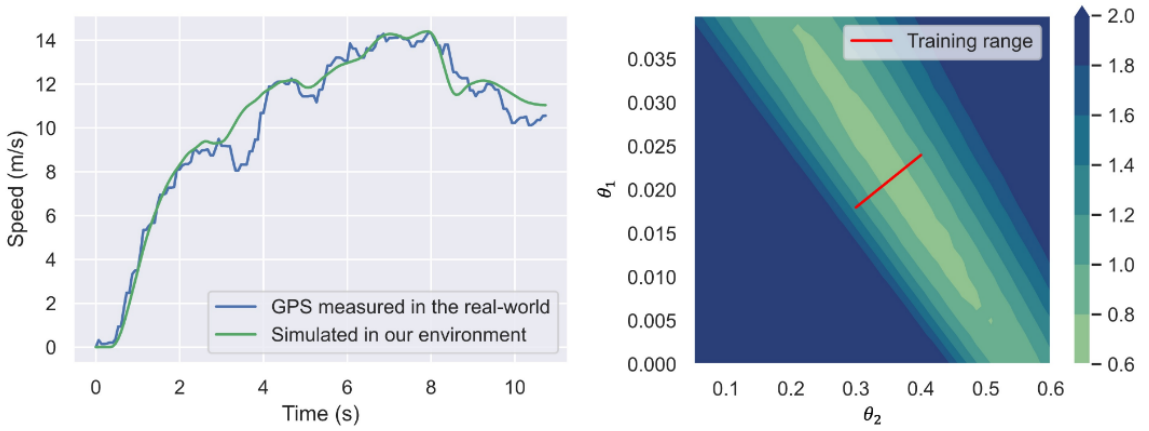


Figure 4.3: TODO This is a placeholder figure, (I need to get the actual data)Velocity response in our sim and gazebo

4.3 Model-Free & Model-Based

4.3.1 Sample Efficency

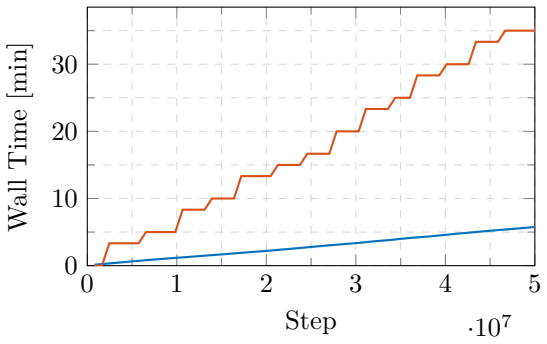


Figure 4.4: TODO

4.3.2 Discrete and Continuous Rewards

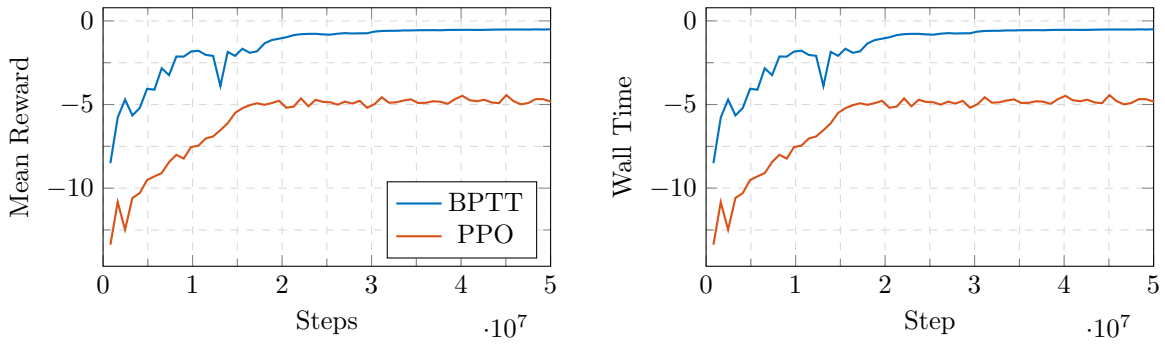


Figure 4.5: TODO

4.3.3 Performance

$$\mathbf{p}(\theta) = (-\text{scale} \cdot \sin(\theta)\cos(\theta),\ -\text{scale} \cdot \cos(\theta),\ \text{h}) \tag{4.6}$$

where $\theta = \omega \cdot t \cdot dt$ and $\omega = \text{speed}/\text{scale}$

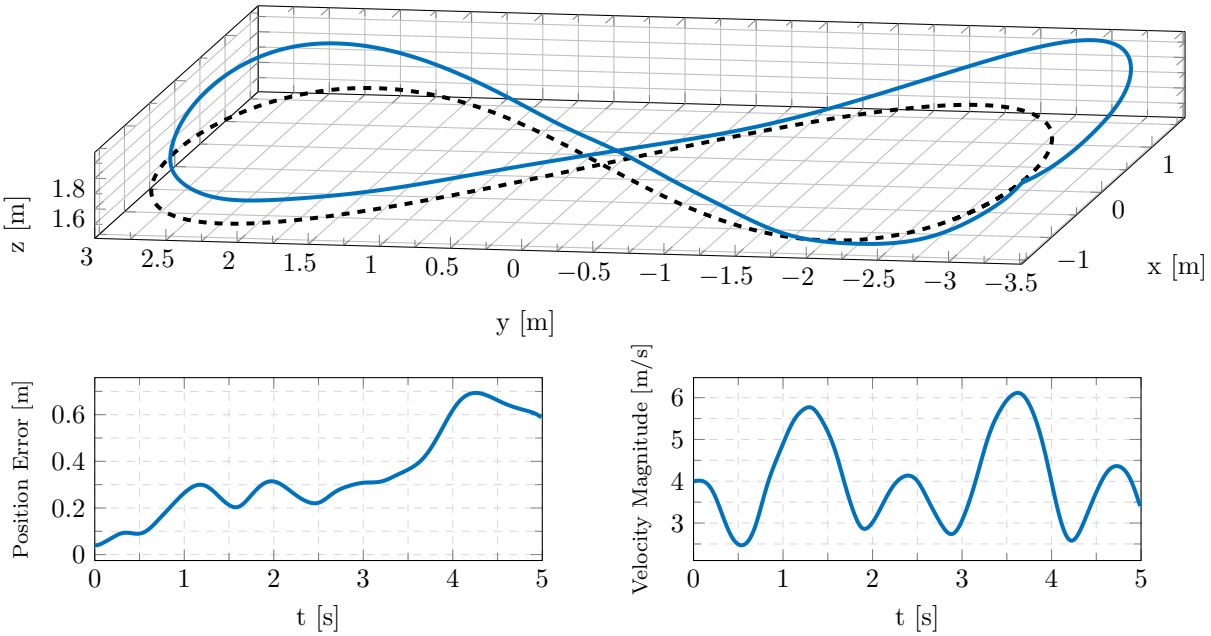


Figure 4.6: TODO

4.3.4 Transferability and Robustness

Table 4.1: Performance comparison of policies trained with differentiable gradients versus PPO, evaluated in the proposed simulator and Gazebo environment.

	BPTT		PPO	
	Ours	Gazebo	Ours	Gazebo
N				
T_t				
SR				
T_f				

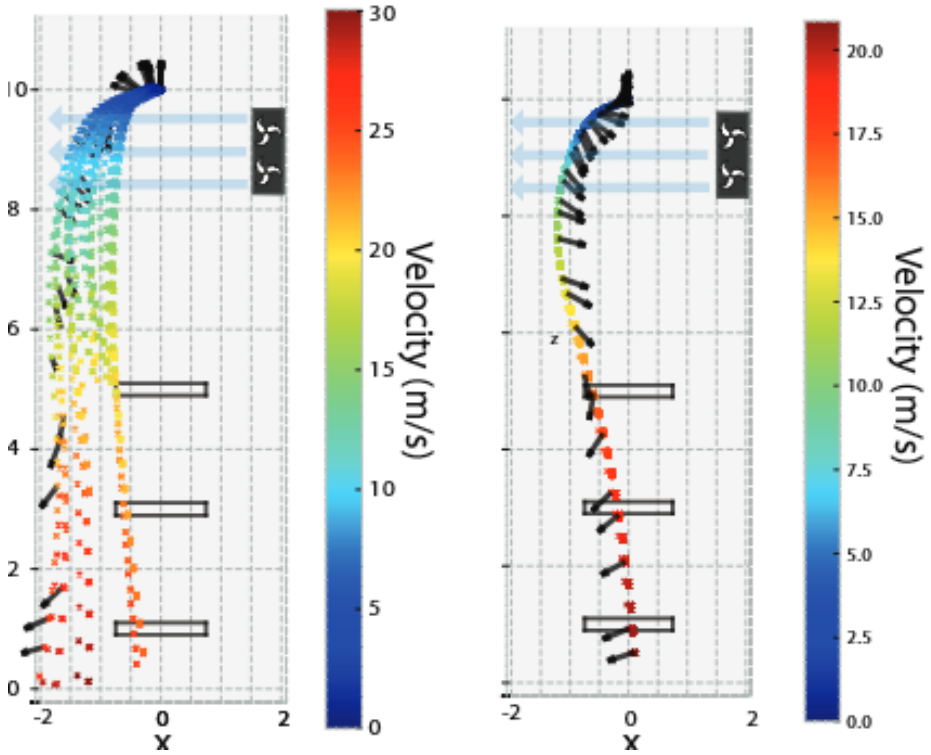


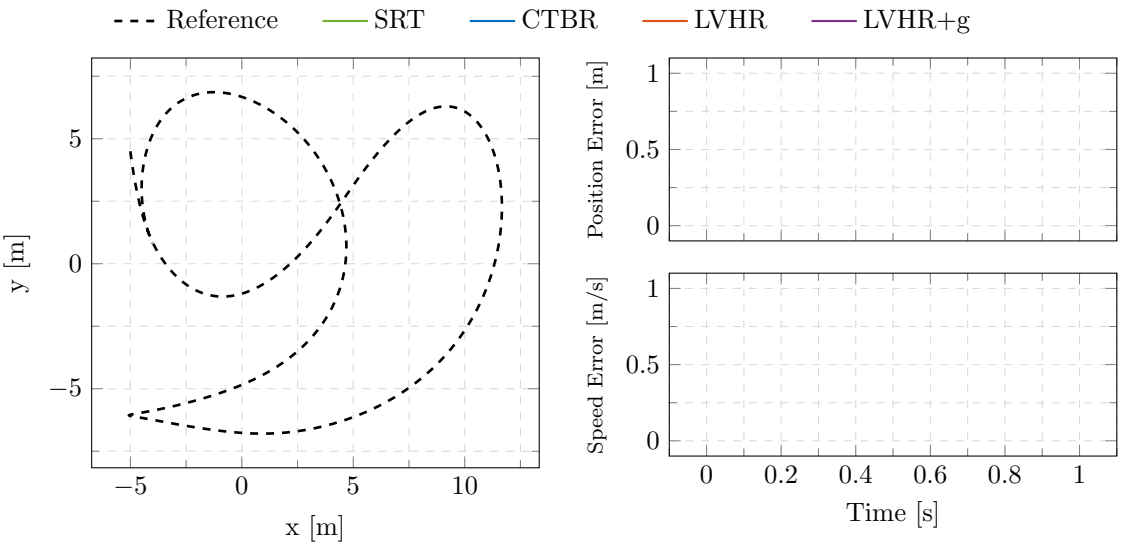
Figure 4.7: TODO This is a placeholder figure, (I need to get the actual data)Wind response, gazebo?

4.4 Control Abstraction

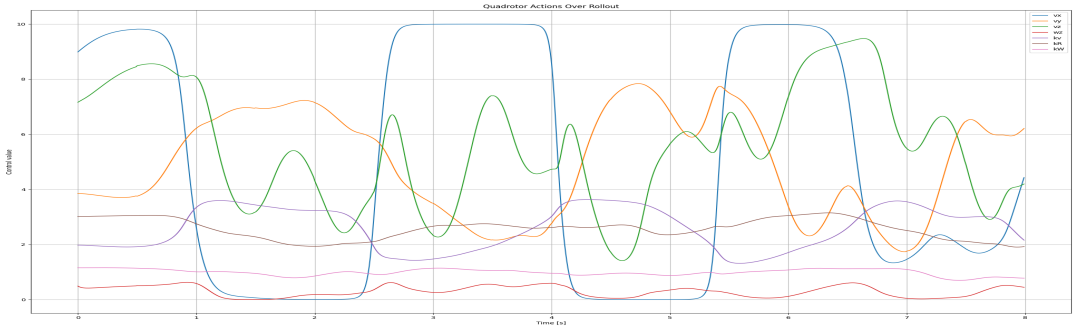
TODO Plot gradients of backward and foorward passes to comare their quality..

4.4.1 Sample Efficency

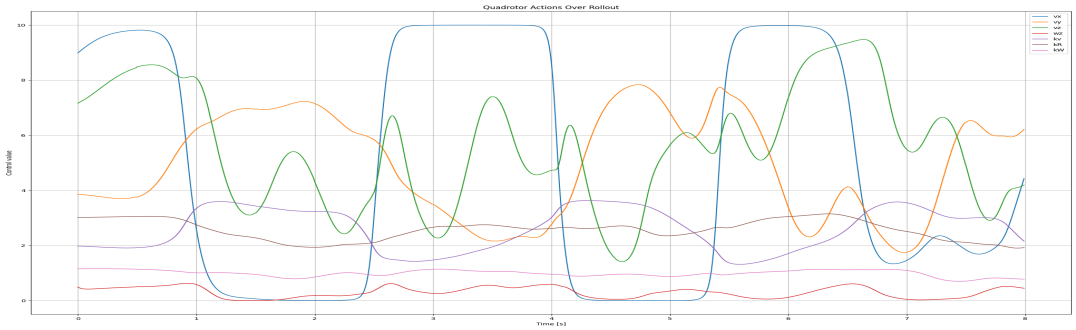
4.4.2 Performance



4.4.3 Transferability and Robustness



(a) TODO Fixed gains



(b) TODO Adaptive gains

Figure 4.8: Adaptive gains comparison

4.5 Real World Validation

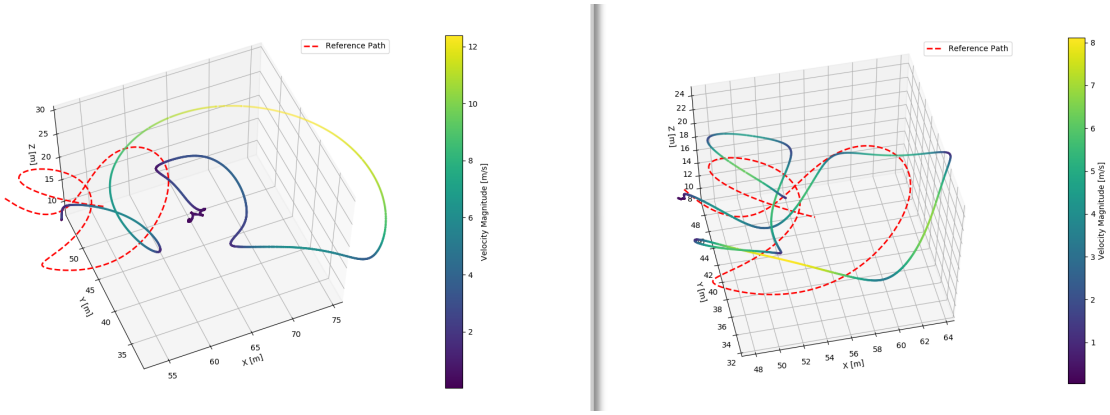


Figure 4.9: TODO This is a placeholder figure, (I need to get the actual data)Real world flights

Chapter 5

Conclusion

Chapter 6

Future Directions

Chapter 7

References

[1] K.J Åström. "Optimal control of Markov processes with incomplete state information". In: *Journal of Mathematical Analysis and Applications* 10.1 (1965), pp. 174–205. ISSN: 0022-247X. DOI: https://doi.org/10.1016/0022-247X6590154-X. URL: https://www.sciencedirect.com/science/article/pii/0022247X6590154X.

[2] Lilian Weng. "Domain Randomization for Sim2Real Transfer". In: *lilianweng/github/io* (2019). URL: https://lilianweng.github.io/posts/2019-05-05-domain-randomization/.

Appendix A

Appendix A